

Mechatronics and Robotics



HOCHSCHULE HEILBRONN

Year: 2024

Semester: Spring

Lecture: Programming 2

Date: 22.07.2024

Prüfer/in: Sarac Heinz, Bayer

ID Number:

Name:

Exam duration: 120 minutes

Allowed material: Handwritten cheat sheet (one A4 page, both sides can be used)

Please wait to open the exam booklet until you are told to do so.

Exam has 3 questions and is worth a total of 100 points if all are answered correctly.

Exam booklet has 8 pages. Make sure to check that all 8 are unique.

Don't forget to write your name and university ID above. If you use extra sheets, write your name and id on these as well.

Question	Points	Score	Notes
1	25		
2	35		
3	40		
Total	max. 100		
	Grade:		
	Date:		Signature

Question 1 (Recursion, 25 points)

Implement the C function

void enumeration(int* bin_num, int length, int offset, int zeros, int ones, int divisor) **recursively** such that it displays all binary numbers which consist of as many zeros as given by the variable **zeros** and as many ones as given by the variable **ones**. Each binary number is stored in an array called **bin_num** from index **offset** to index **length – 1**, i.e. from **bin_num[offset]** to **bin_num[length – 1]**. It should only display the numbers if they are divisible by the parameter **divisor**. You can use the provided function **void display(int* bin_num, int length)** to print out the binary number from an array of length **length**. Furthermore, you can use the function **int divisible(int* bin_num, int length, int divisor)** to check whether the binary number represented by the **bin_num** array with length **length** is divisible by **divisor**.

Implement the C code such that it is correct for arbitrary valid definitions of zeros, ones, and divisor.

You can assume that for the initial call of **enumeration**, the following conditions hold: $\text{length} = \text{zeros} + \text{ones}$, $\text{length} > 0$, $\text{offset} = 0$, $\text{zeros} \geq 0$, $\text{ones} \geq 0$ and $\text{divisor} > 0$.

If you don't implement the function recursively, you will not receive any points. Explain your approach shortly.

Sample output 1 (zeroes = 3, ones = 2, divisor = 4):

11000
10100
01100

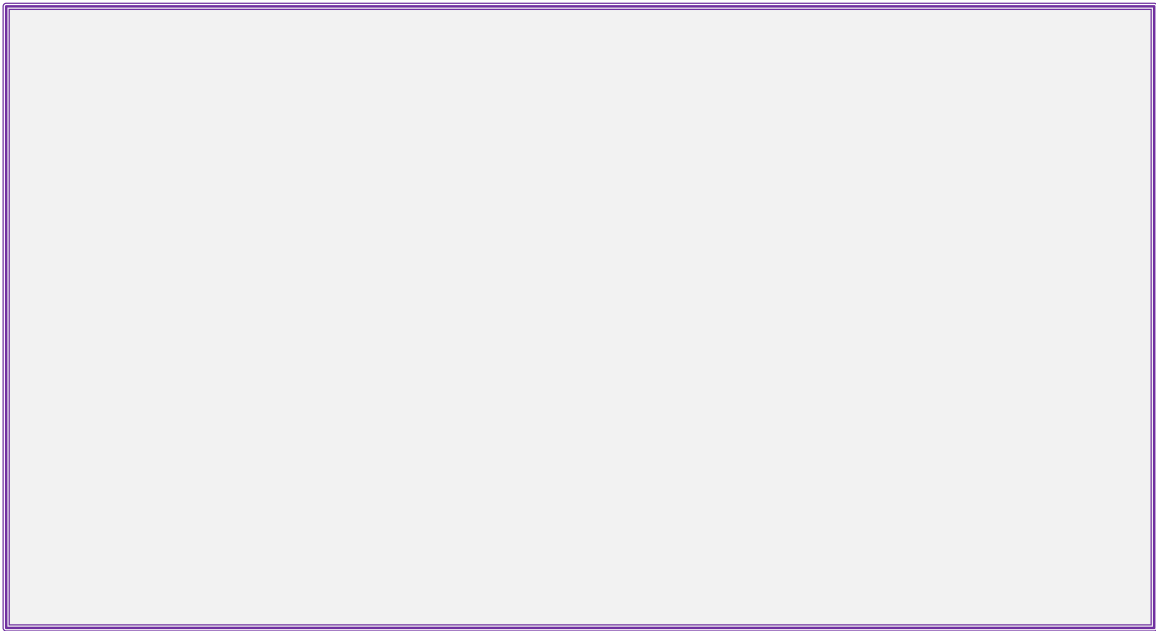
Sample output 2 (zeroes = 4, ones = 2, divisor = 6):

110000
011000
100100
001100
010010
000110

```
#include <stdio.h>

int divisible(int* bin_num, int length, int divisor){
    int i, power, int_num;
    power = 1;
    int_num = 0;
    for(i = 0; i < length; i++){
        int_num += bin_num[i] * power;
        power *= 2;
    }
    return int_num % divisor == 0;
}

void display(int* bin_num, int length){
    int i;
    for(i = length - 1; i >= 0; i--){
        printf("%d", bin_num[i]);
    }
    printf("\n");
}

void enumeration(int* bin_num, int length, int offset, int zeros,
                int ones, int divisor){
    
}

int main(){
    int zeros = 3;
    int ones = 4;
    int divisor = 5;
    int bin_num[zeros + ones];
    enumeration(bin_num, zeros + ones, 0, zeros, ones, divisor);
    return 0;
}
```

Question 2 (Linked Lists, 35 points)

Implement the C function

struct node * insert_between(struct node * header, int value, int first_value, int second_value) that modifies the singly linked list pointed to by **header** such that it inserts a new node with value **value** between two nodes each time a node with value **first_value** is followed by a node with value **second_value**. Don't create a new list but use the existing one. If **header** is NULL then the list is empty, otherwise **header** points to the first element of the list. Make sure that your function terminates under all conditions, in particular the one depicted in sample output 2.

Implement the C code such that it is correct for arbitrary valid singly linked lists and parameters.

Explain your implementation approach shortly.

Sample output 1 (`list = insert_between(list, 6, 3, 2)`):

List: 3 2 3 6 3 2

List: 3 6 2 3 6 3 6 2

Sample output 2 (`list = insert_between (list, 3, 3, 2)`):

Liste: 3 2 3 6 3 2

Liste: 3 3 2 3 6 3 3 2

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    int value;
    struct node* next;
};

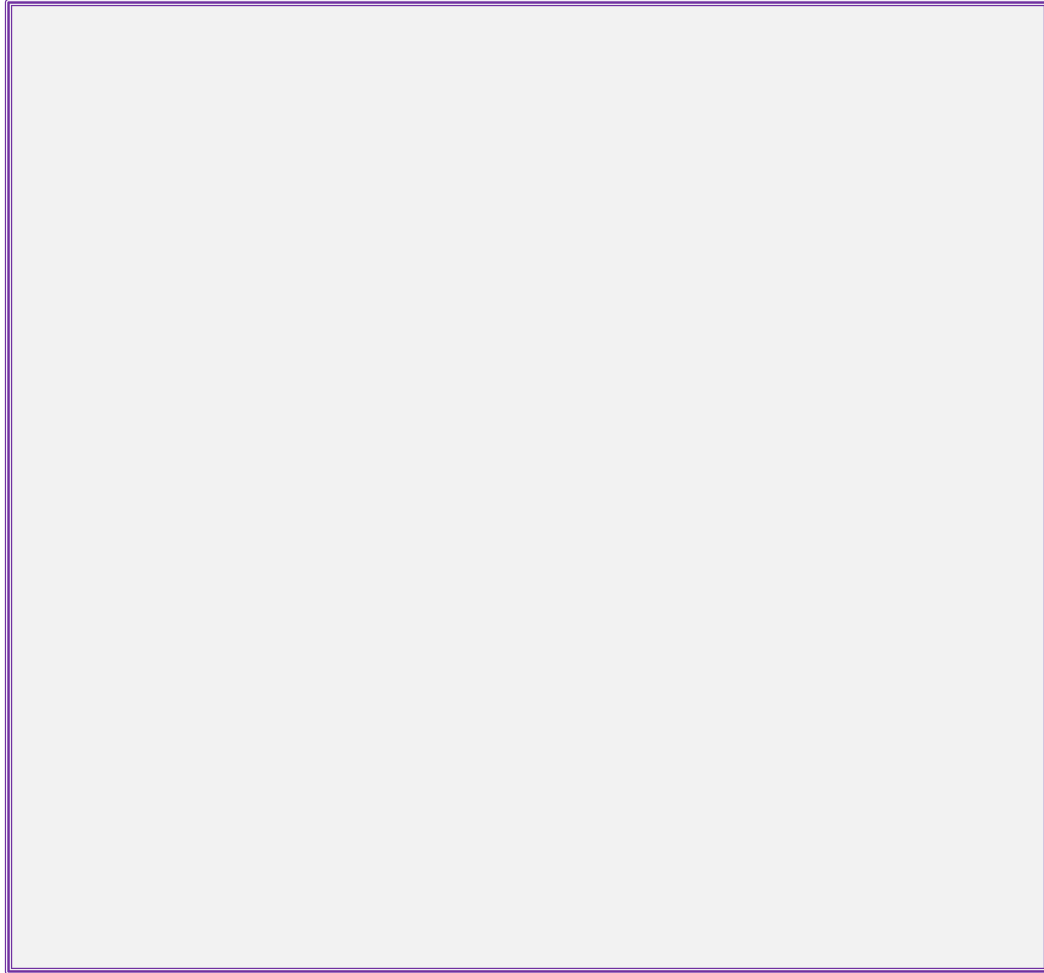
void display(struct node * header){
    struct node * pt = header;
    while(pt != NULL){
        printf("%d ", pt->value);
        pt = pt->next;
    }
    printf("\n");
}

struct node* append(struct node* header, int value){
    struct node * newElement;
    struct node * pt = header;

    newElement = (struct node *) malloc(sizeof(*newElement));
    if(newElement==NULL){
        printf("Not enough memory.\n");
        return header;
    }
    newElement->value = value;
    newElement->next = NULL;

    if(pt == NULL) { //empty List
        return newElement;
    }
    while(pt->next != NULL){
        pt = pt->next;
    }
    pt->next = newElement;
    return header;
}
```

```
struct node * insert_between(struct node * header, int value,  
                             int first_value, int second_value){
```



```
}
```

```
int main(){  
struct node *list;  
    list = NULL;  
    list = append(list, 3);  
    list = append(list, 2);  
    list = append(list, 3);  
    list = append(list, 6);  
    list = append(list, 3);  
    list = append(list, 2);  
  
    printf("List: ");  
    display(list);  
  
    list = insert_between(list, 6, 3, 2);  
    printf("List: ");  
    display(list);  
    return 0;  
}
```

Question 3 (Complex Data Types, 40 points)

Implement the C function

void sort_times(struct time *t, int length) such that the sequence of times stored in an array of time structures **t** with length **length** is sorted in ascending order. The type struct **time** characterizes a given time by hour, minute and an hour offset given in coordinated world time (UTC). For instance, the three following times are identical: 14:00 with UTC offset 0 (e.g. London), 17:00 with UTC offset 3 (e.g. Moscow) and 9:00 with UTC offset -5 (e.g. New York City)

Furthermore, implement the C function

void search_times(struct time *times, int length, struct time now, int duration) such that it outputs all the times from the array of structs **times** with length **length** in ascending order that lie within a specified time interval. This interval is characterized by its starting time given by the parameter **now** and its duration given in minutes by the parameter **duration**. You may call the function **sort_times**. Assume that **length** and **duration** are non-negative. The choice of the sorting algorithm is up to you.

Implement the C code such that it is correct for arbitrary parameters and not only for the given examples.

Explain your implementation approach shortly.

Sample Output

Time: 11:55, UTC: 3

Time: 12:58, UTC: 4

Time: 7:10, UTC: -2

Time: 8:15, UTC: -1

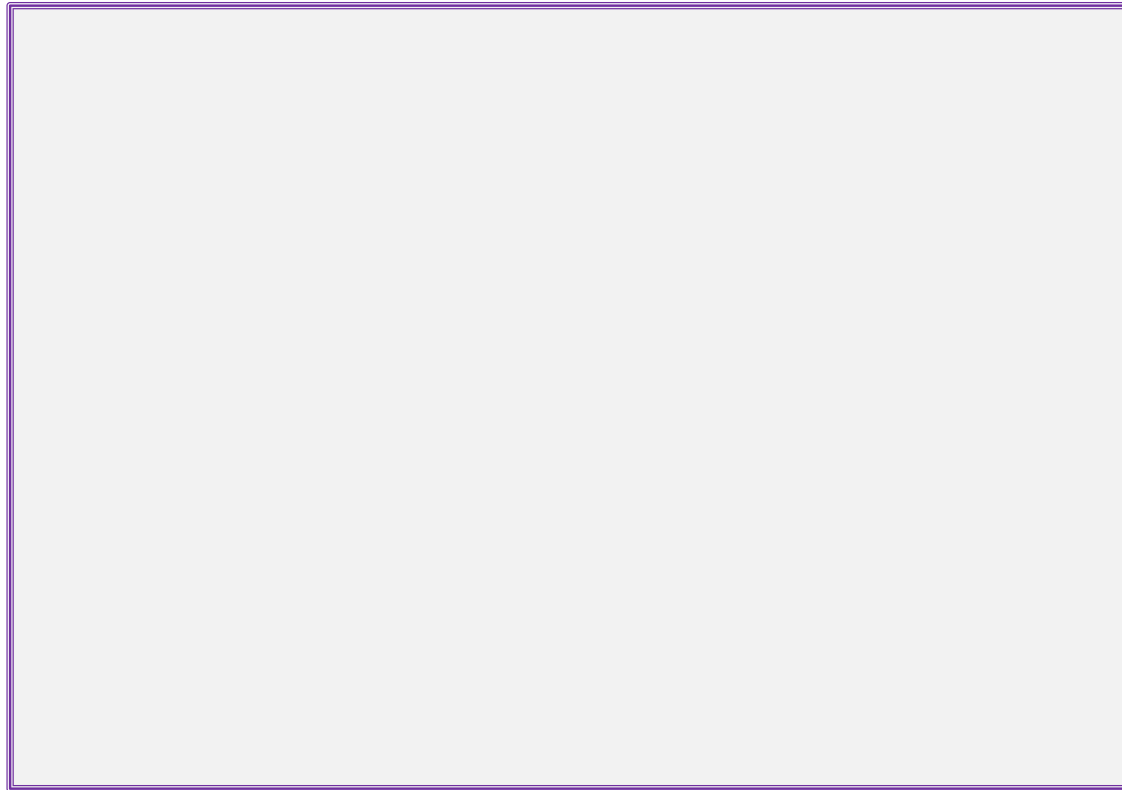
```
#include <stdio.h>
#include <stdlib.h>
```

```
struct time{
    int hour;
    int minute;
    int utc_offset;
};
```

```
void sort_times(struct time *t, int length){
```

```
}
```

```
void search_times(struct time *times, int length,  
                 struct time now, int duration){
```



```
}
```

```
int main(){  
    struct time times[] = {  
        {12, 58, 4}, {11, 55, 3}, {10, 15, 0},  
        {15, 0, 9}, {11, 30, 10}, {12, 15, -9},  
        {19, 46, 11}, {7, 10, -2}, {8, 15, -1}  
    };  
    int length = 9;  
  
    struct time now = {10, 55, 2};  
    int duration = 30;  
  
    search_times(times, length, now, duration);  
  
    return 0;  
}
```